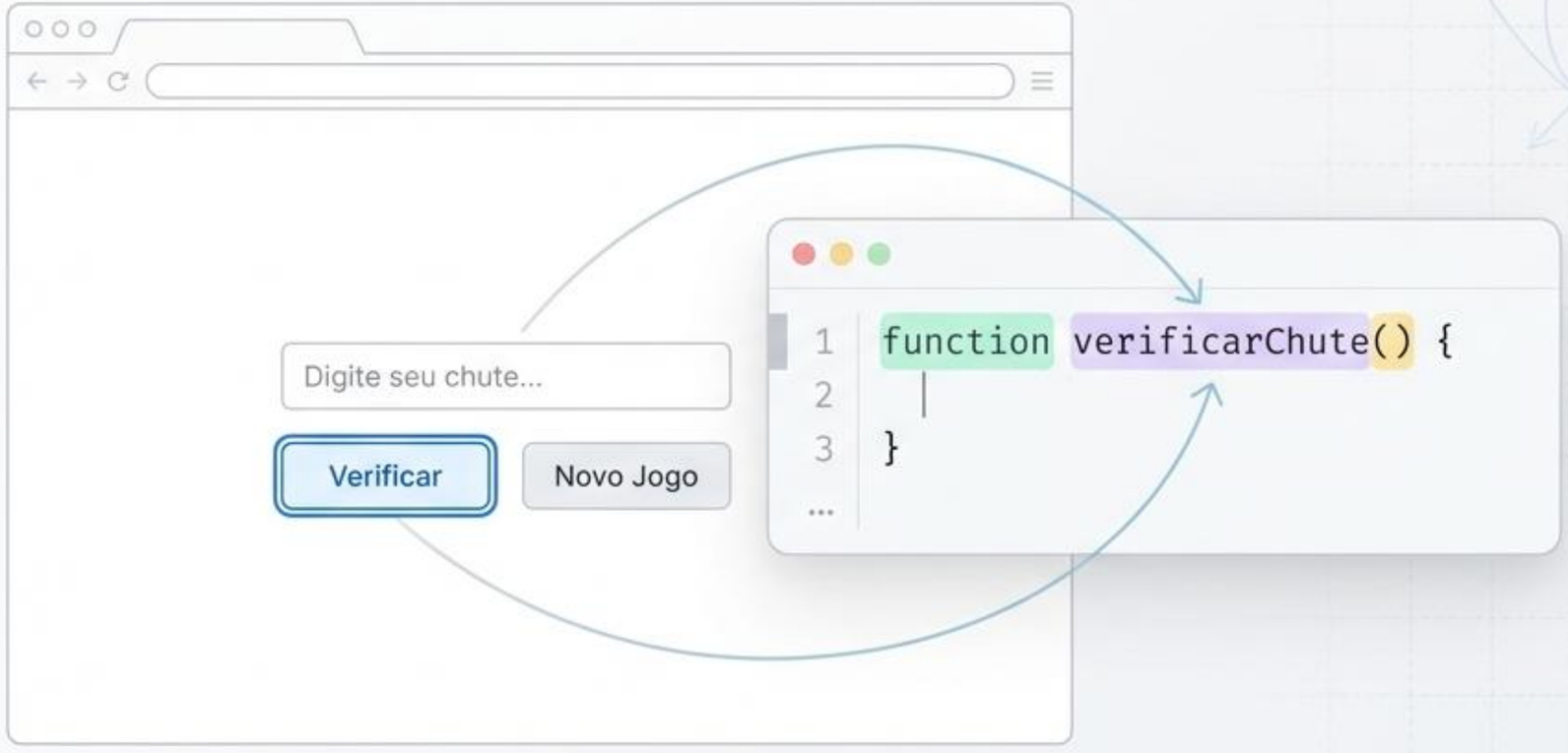
Desenvolvimento Front-End II: Lógica com JavaScript

Guia Prático: Estruturando o Jogo do Número Secreto



Do HTML estático à interatividade dinâmica.

NotebookLM

Como o JavaScript Encontra e Modifica o HTML

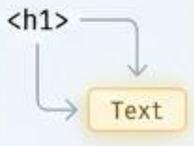
O escopo: representa a página inteira.

A ferramenta de busca: localiza o primeiro elemento que corresponda ao seletor.

```
document.querySelector('tag').innerHTML = 'texto';
```

```
<h1>Titulo</h1>
<p>Parágrafo</p>
<button>Clique</button>
```

O alvo: a propriedade HTML (ex: h1, p, button). Identifica o que será alterado (Case Sensitive).



A ação: acessa e substitui o conteúdo interno do elemento HTML selecionado.

NotebookLM

Injetando Textos Iniciais na Interface do Jogo

Wireframe Mapping

Jogo do número secreto

Escolha um número entre 1 e 10

Project Code

```
1
2  let titulo = document.querySelector('h1');
3  titulo.innerHTML = 'Jogo do número secreto';
4
5  let paragrafo = document.querySelector('p');
6  paragrafo.innerHTML = 'Escolha um número entre 1 e 10';
7
```

Key Takeaway: O uso de variáveis (**let**) armazena a seleção do DOM, permitindo a injeção de strings de texto diretamente na interface sem alterar o arquivo .html.

NotebookLM

A Anatomia de uma Função e seu **Gatilho** (Eventos)

O atributo HTML escuta o clique do usuário.

```
<button onclick="nomeDaFuncao()">
```

Chama a execução

Declaração

```
function nomeDaFuncao() {
  // Escopo: Responsabilidade única da função
}
```

Corpo da função

NotebookLM

Conectando o Botão Chutar à Lógica do Projeto

```
<button onclick="verificarChute()" class="container__botao">Chutar</button>
```

Chutar

JavaScript Function

```
1  function verificarChute() {
2      console.log('O botão foi clicado!');
3  }
```

```
> O botão foi clicado!
> O botão foi clicado!
> O botão foi clicado!
```

A função `verificarChute()` possui uma única responsabilidade inicial: capturar o evento de clique e confirmar a comunicação via console (ferramenta de diagnóstico do desenvolvedor).

NotebookLM

Evitando Repetição de Código com Parâmetros

O Problema - **Código Estático**

```
let titulo = document.querySelector('h1');
titulo.innerHTML = 'Jogo do número secreto';
titulo.innerHTML = 'Jogo do número secreto';

let paragrafo = document.querySelector('p');
paragrafo.innerHTML = 'Escolha um número entre
1 e 10';
```

A Solução - **Código Dinâmico**

```
function exibirTextoNaTela(tag, texto) {
    let campo = document.querySelector(tag);
    campo.innerHTML = texto;
}
```

Parâmetros (`tag`, `texto`) atuam como variáveis temporárias que recebem valores diferentes a cada chamada da função.

NotebookLM

Refatorando a Exibição de Textos no Jogo

```
1 exibirTextoNaTela('h1', 'Jogo do número secreto');
2 exibirTextoNaTela('p', 'Escolha um número entre 1 e 10');
```

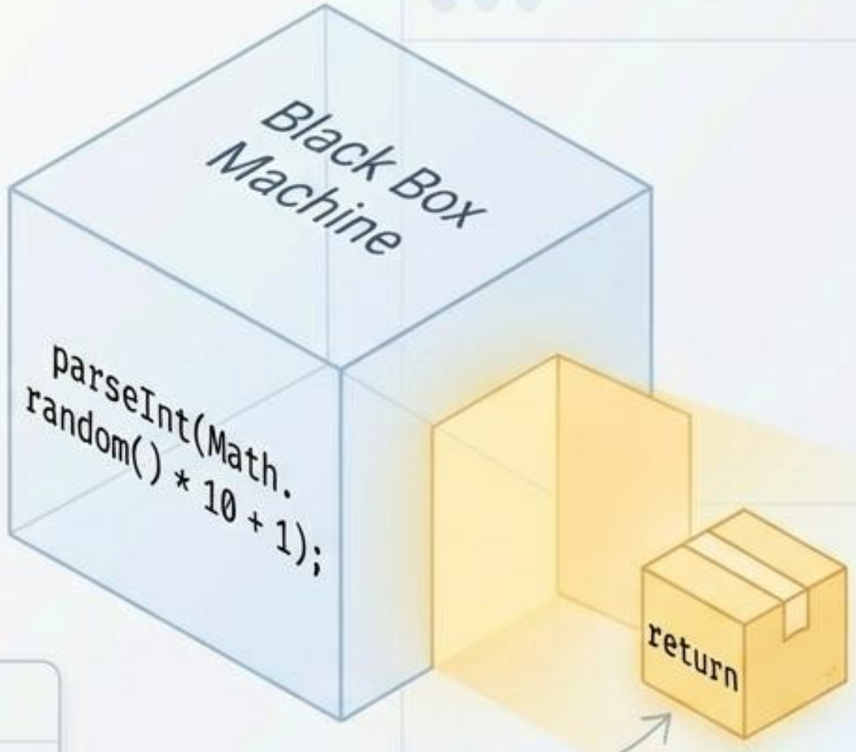
Jogo do número secreto

Escolha um número entre 1 e 10

Ao isolar o comportamento de exibição em uma única função, o escopo global fica limpo, legível e altamente escalável para futuras mensagens.

NotebookLM

Entregando Resultados com a Instrução "return"



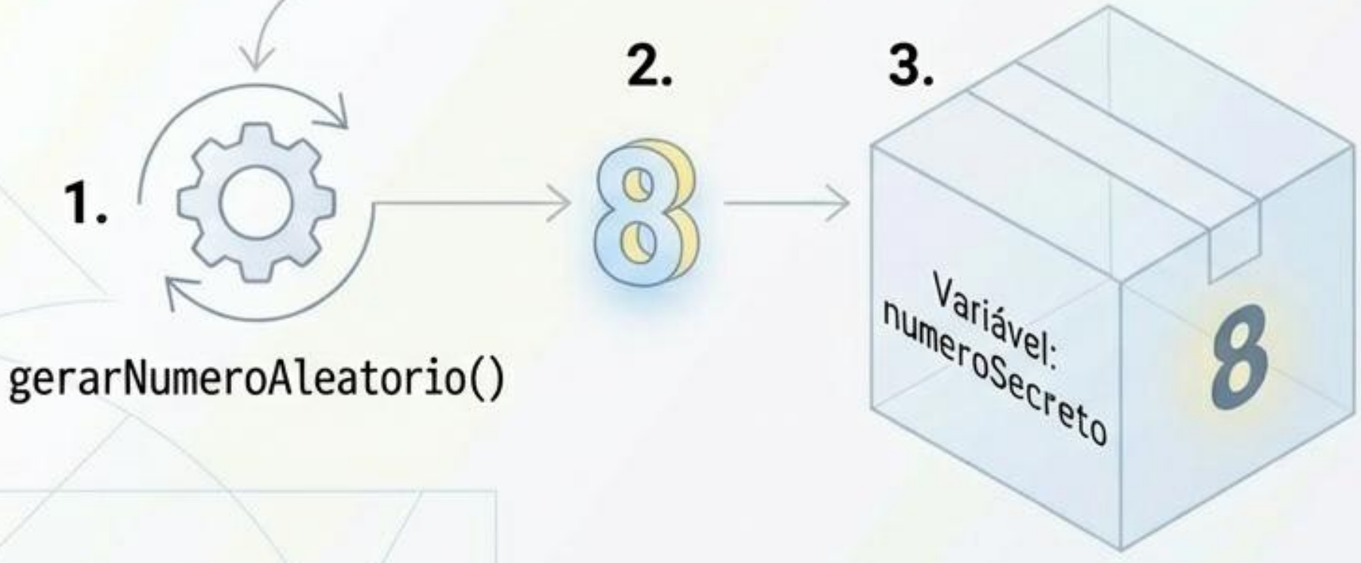

```
1  function gerarNumeroAleatorio() {
2      return parseInt(Math.random() * 10 + 1);
3  }
```

A palavra-chave `return` encerra a execução da função e devolve o valor processado para quem a invocou.

NotebookLM

Gerando e Armazenando o Número Secreto

```
1  let numeroSecreto = gerarNumeroAleatorio();
```



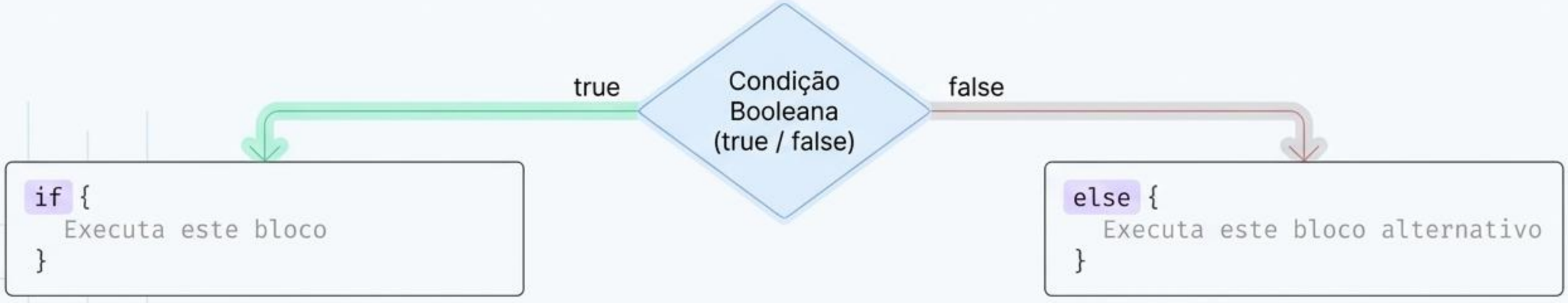
1. O código chama a função **gerarNumeroAleatorio()**.
2. A função calcula o número (ex: 8) e utiliza o **return**.
3. O valor retornado é capturado e permanentemente armazenado na variável global **numeroSecreto**.

```
> 8
```

NotebookLM

Operadores Relacionais e Lógica Condicional

	Operador	Nome	Função
1	`=`	Atribuição	Guarda um valor na memória.
2	`==`	Comparação	Verifica igualdade entre dois valores.



Booleanos: O núcleo lógico da programação, retornando exclusivamente verdadeiro (true) ou falso (false).

NotebookLM

Avaliando o Chute do Usuário

```
let chute = document.querySelector('input').value;
```

Extrai apenas o dado numérico digitado, ignorando as tags HTML da estrutura.

```
if (chute == numeroSecreto) {
  exibirTextoNaTela('h1', 'Acertou!');
} else {
  // Lógica de erro (maior ou menor)
}
```

NotebookLM

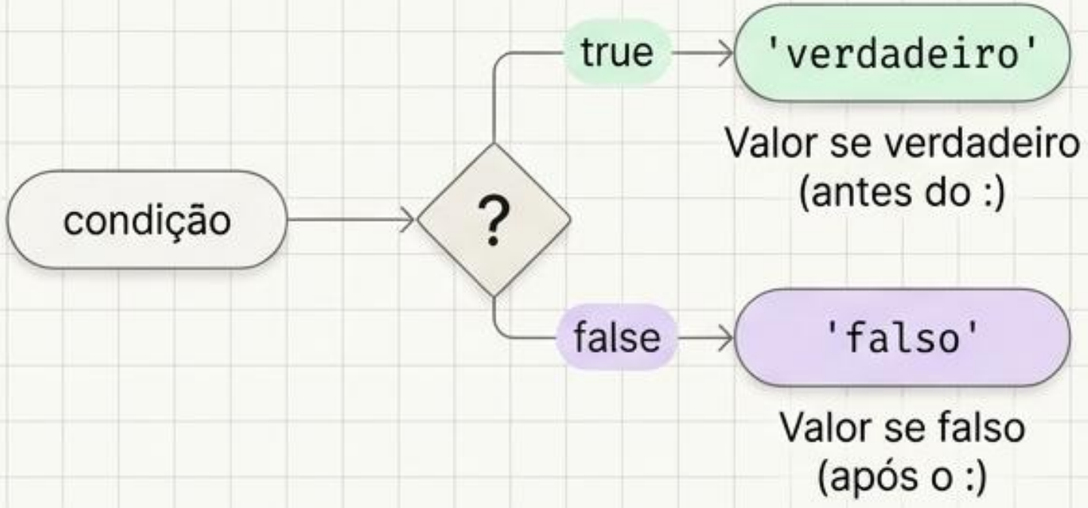
Template Strings e o Operador Ternário

```
``Texto ${variavel}``
```

Crases:
Enquadram a string.

Interpolação: Injeta JavaScript diretamente dentro de uma string de texto sem usar sinal de adição.

```
condição ? 'verdadeiro' : 'falso'
```



Ferramentas avançadas que reduzem múltiplas linhas de código (if/else tradicionais) em lógicas precisas de uma única linha.

NotebookLM

Contabilizando Tentativas Dinamicamente

Step 1: Incremento

```
tentativas++
```

→ Ocorre a cada erro.

Step 2: Decisão Gramatical

```
let palavra = tentativas > 1 ? 'tentativas' : 'tentativa';
```

↳ Determina a forma plural ou singular com base na contagem.

Step 3: Interpolação

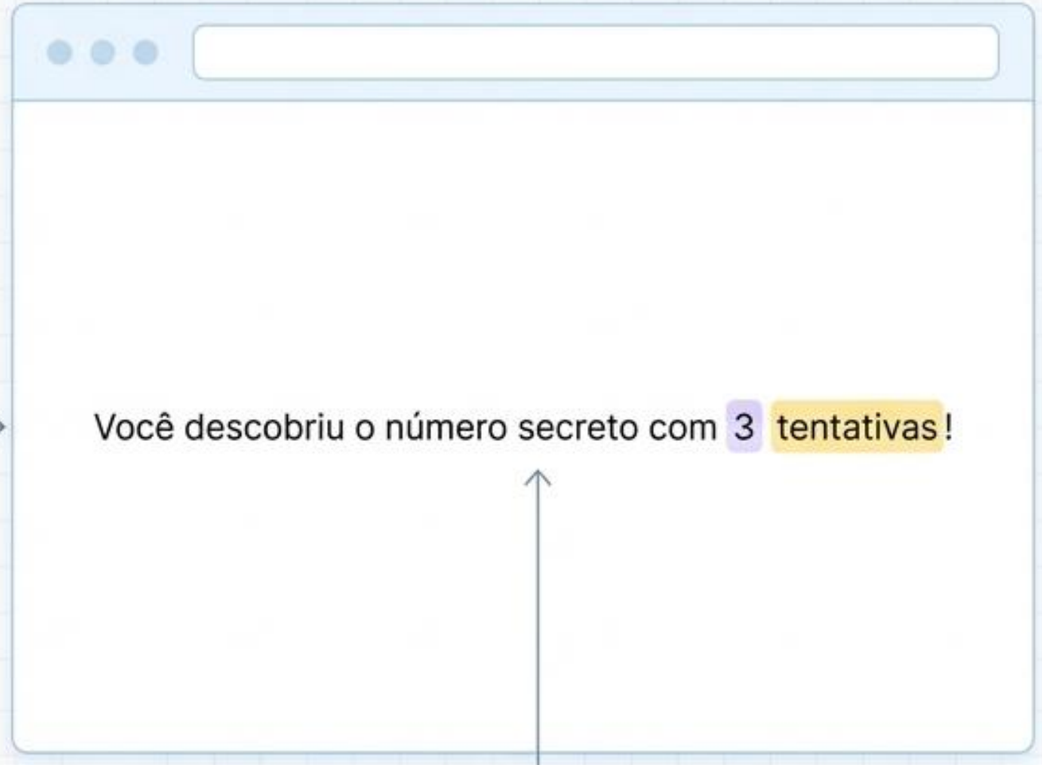
```
let mensagem = `Você descobriu o número secreto com
${tentativas} ${palavra}!`;
```

↳ Gera uma string dinâmica combinando texto e variáveis.

Step 4: Exibição

```
exibirTextoNaTela('p', mensagem);
```

↳ Invoca a função para mostrar o resultado no elemento <p>.



NotebookLM

Buscando por ID e Manipulando Atributos HTML

Targeting Rule

Seletor Geral	Seletor Específico por ID
`querySelector('.classe / tag')`	`getElementById('nome_do_id')`
Seleciona o primeiro elemento que corresponde a um seletor CSS.	getElementById localiza um identificador único no HTML, ignorando a sintaxe de seletores CSS.

State Manipulation Visualizer

```
document.getElementById('id_unico').removeAttribute('disabled');
```

Antes:

removeAttribute('disabled') remove o atributo, desbloqueando o elemento.

Depois:

NotebookLM

Reiniciando o Ciclo do Jogo

Ação 1: Limpando o Input

```
let chute = document.querySelector('input');
chute.value = '';
```

Ação 2: Habilitando o Botão de Reinício

```
document.getElementById('reiniciar').
removeAttribute('disabled');
```

Ao manipular propriedades dinâmicas (.value) e atributos rígidos (disabled), controlamos com precisão o fluxo de vida e o estado da aplicação web.

NotebookLM

Para saber mais: O poder das Funções no JavaScript

Se o HTML é o esqueleto e o CSS é a estética, as funções são, sem dúvida, os músculos e o cérebro do seu código. Elas permitem que você encapsule blocos de lógica que podem ser reutilizados em qualquer parte do projeto, evitando que você precise escrever a mesma coisa várias vezes. No desenvolvimento de software moderno, entender a anatomia das funções é o que separa um iniciante de um desenvolvedor que escreve código elegante e eficiente.

Os Diferentes Estilos de Funções

O JavaScript é uma linguagem extremamente flexível, e isso se reflete na maneira como podemos declarar funções. Dependendo do que você precisa — se a função deve receber uma informação de fora ou se ela deve devolver um resultado calculado — ela pode assumir diferentes formas:

1. **A Função Solidária (Sem retorno e sem parâmetro):** É a forma mais simples. Ela apenas executa uma tarefa fixa sempre que é chamada. Imagine um botão que apenas limpa a tela; ela não precisa saber quem você é nem te devolver um valor, apenas agir.
2. **A Função Atenciosa (Sem retorno e com parâmetro):** Esta função está pronta para agir, mas precisa de um detalhe seu para começar. Por exemplo, uma função `cumprimentar(nome)` precisa do seu nome para exibir uma mensagem personalizada na tela.
3. **A Função Calculadora (Com retorno):** Aqui as coisas ficam interessantes. Estas funções processam dados e "te entregam" um resultado. Elas podem ser autossuficientes (gerar um número aleatório sem precisar de entrada) ou dependentes (receber dois números e retornar a soma deles). O segredo aqui é a palavra-chave `return`, que faz o valor "sair" da função para ser guardado em uma variável.

O Salto para a Modernidade: Funções Anônimas e Arrow Functions

À medida que você avança na programação, encontrará formas mais enxutas e modernas de escrever lógica.

As **Funções Anônimas** são aquelas que não possuem um nome próprio e geralmente são guardadas dentro de uma variável. Elas são muito úteis quando queremos que uma função seja tratada como um dado qualquer dentro do sistema.

Já as **Arrow Functions** (funções de seta) são a evolução sintática do JavaScript moderno. Elas permitem escrever funções de forma extremamente curta e direta. Por exemplo, calcular o quadrado de um número pode ser feito em apenas uma linha: `let quadrado = x => x * x;`. Além de deixarem o código mais limpo, elas resolvem questões técnicas avançadas sobre como o JavaScript entende o contexto do código.

Por onde começar?

Não se sinta pressionado a decorar todos esses tipos agora. Na jornada de um desenvolvedor, o segredo é o mergulho gradual. Comece com as funções tradicionais para entender a lógica e, conforme sentir necessidade de organizar melhor sistemas maiores, essas variações surgirão naturalmente como soluções para seus problemas.

Lembre-se: uma função bem escrita é como uma peça de LEGO; ela se encaixa perfeitamente no seu projeto e faz com que tudo funcione de maneira mais dinâmica e interativa.

Para saber mais: as diferenças entre HTML, CSS e JavaScript

No vasto universo do desenvolvimento web, existem três tecnologias que se destacam como os pilares de tudo o que vemos e interagimos no navegador. Compreender como o HTML, o CSS e o JavaScript se complementam é o primeiro passo para qualquer pessoa que deseja criar experiências digitais memoráveis.

O Esqueleto: HTML

O **HTML (HyperText Markup Language)** é a espinha dorsal de qualquer página. Criado por Tim Berners-Lee em 1991, ele não é uma linguagem de programação, mas sim uma linguagem de marcação. Pense nele como o esqueleto de um edifício: ele define onde estão os pilares, as paredes e as portas. Através de "tags", o HTML organiza o conteúdo — títulos, parágrafos, imagens e links — garantindo que cada elemento tenha seu lugar e sua hierarquia bem definidos. Sem o HTML, a web seria apenas um emaranhado de dados sem estrutura.

A Estética: CSS

Se o HTML é o esqueleto, o **CSS (Cascading Style Sheets)** é o acabamento, a pintura e a decoração. Lançado em 1995, o CSS veio para separar a estrutura da apresentação. É ele quem decide que um título deve ser azul, que um parágrafo deve ter uma fonte específica ou que o layout deve se ajustar perfeitamente tanto em um monitor gigante quanto na tela de um celular (o que chamamos de design responsivo). O CSS permite que a criatividade flua, transformando documentos estruturados em interfaces visualmente atraentes e profissionais.

A Inteligência: JavaScript

Por fim, temos o **JavaScript (JS)**, a única linguagem de programação desse trio. Ele é o responsável pelo "cérebro" da aplicação. É através do JavaScript que adicionamos vida e dinamismo: animações que reagem ao mouse, validações de formulários em tempo real e atualizações de conteúdo sem que a página precise ser recarregada. Ele capacita o desenvolvedor a criar interatividade, tornando a experiência do usuário fluida e inteligente.

Por que dominar essas três tecnologias?

Dominar esse conjunto não é apenas uma questão técnica, é uma questão estratégica. Entender o HTML profundamente permite criar sites acessíveis e bem indexados por motores de busca (SEO). Dominar o CSS garante que sua ideia seja visualizada exatamente como você imaginou em qualquer dispositivo. E a proficiência em JavaScript abre as portas para criar funcionalidades avançadas que encantam o usuário final.

Atualmente, o mercado de trabalho valoriza imensamente profissionais que compreendem essa base. Em um mundo onde a presença online é vital para empresas e indivíduos, saber materializar ideias de forma funcional e criativa é um investimento de carreira com retorno garantido. O aprendizado contínuo dessas ferramentas é o que diferencia um desenvolvedor comum de um profissional de excelência.

```html
<!DOCTYPE html>
<html lang="pt-br">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://code.responsivevoice.org/responsivevoice.js"></script>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Chakra+Petch:wght@700&family=Inte
r:wght@400;700&display=swap"
        rel="stylesheet">
    <link rel="stylesheet" href="style.css">
    <title>JS Game</title>
</head>

<body>

    <div class="container">
        <div class="container__conteudo">
            <div class="container__informacoes">
                <div class="container__texto">
                    <h1></h1>
                    <p class="texto__paragrafo"></p>
                </div>
                <input type="number" min="1" max="10" class="container__input">
                <div class="chute container__botoes">
```

```
                <button onclick="verificarChute()"
class="container__botao">Chutar</button>

                <button onclick="" id="reiniciar" class="container__botao" disabled>Novo
jogo</button>

            </div>

          </div>

          <img src="./img/ia.png" alt="Uma pessoa olhando para a esquerda"
class="container__imagem-pessoa" />

        </div>

    </div>



    <script src="app.js" defer></script>

</body>


</html>
```

```
function exibeMensagemInicial(tag, texto){
   let campo = document.querySelector(tag);
   campo.innerHTML = texto;
}

let tentativas = 1;

function msgInicio(){
exibeMensagemInicial('h1', 'Jogo do Numero Secreto');
exibeMensagemInicial('p', 'Escolha um numero entre 1 e 10');
}

msgInicio()

numeroSecreto = gerarNumeroSecreto();

function gerarNumeroSecreto(dificuldade){
   return parseInt(Math.random() * dificuldade + 1);
}

function verificarChute() {
   let chute = document.querySelector('input').value;
      if (chute == numeroSecreto){
         exibeMensagemInicial('h1', 'Acertou');
         let palavraTentativa = tentativas == 1 ? 'tentativa':'tentativas';
         let msgTentativa = `Você acertou o número secreto (${numeroSecreto}), com
${tentativas} ${palavraTentativa}`
```

```
        exibeMensagemInicial('p', msgTentativa);
        document.getElementById('reiniciar').removeAttribute('disabled');
    } else{
        tentativas++;
        if (chute > numeroSecreto){
            exibeMensagemInicial('p', 'O número secreto é Menor');
        }else{
            exibeMensagemInicial('p', 'O número secreto é Maior');
        }
        limparCampo()
    }
}

function limparCampo() {
    chute = document.querySelector('input');
    chute.value = '';
}

function novoJogo(){
    msgInicio();
    numeroSecreto = gerarNumeroSecreto();
    tentativas = 1;
    limparCampo();
    document.getElementById('reiniciar').setAttribute('disabled', true);
}

function definirDificuldade(){
    return prompt('Escolha o número limite para o número secreto');
```

}

Guia de Desenvolvimento: Interatividade e Lógica com JavaScript

No desenvolvimento Front-end, a capacidade de tornar uma página estática em uma aplicação dinâmica é o que define um software de qualidade. Este material explora como o JavaScript interage com o HTML para criar o jogo do "Número Secreto", abordando desde a manipulação de elementos até a estruturação de funções complexas.

1. A Ponte entre HTML e JavaScript: O DOM

Para que o JavaScript consiga "enxergar" e alterar o que está na tela, ele utiliza uma interface chamada DOM (*Document Object Model*). No projeto do Número Secreto, abandonamos o uso de comandos simples como `alert` e `prompt` para manipular diretamente os elementos da página.

Seleção de Elementos

O comando fundamental para capturar um fragmento do HTML é o `document.querySelector()`.

- **Sintaxe:** `let titulo = document.querySelector('h1');`
- **Observação Técnica:** O JavaScript é *case sensitive*. Portanto, o "S" de `querySelector` deve ser sempre maiúsculo.

Alterando o Conteúdo

Uma vez selecionado o elemento, utilizamos a propriedade `.innerHTML` para inserir ou modificar o texto.

- **Analogia:** Se o HTML é a moldura de um quadro, o `.innerHTML` é a pintura que colocamos dentro dela.

2. Funções: Organização e Reutilização

Uma função é um bloco de código desenhado para executar uma tarefa específica. Elas são essenciais para evitar a repetição de código (princípio DRY - *Don't Repeat Yourself*).

Tipos de Funções Exploradas:

1. **Sem Parâmetros e Sem Retorno:** Executam uma ação fixa, como o `limparCampo()`.
2. **Com Parâmetros:** Ganham flexibilidade. Exemplo: `exibirTextoNaTela(tag, texto)`. Aqui, a função decide qual tag alterar baseada no que você "passa" para ela.
3. **Com Retorno:** Calculam um valor e o devolvem para quem a chamou. Essencial para a função `gerarNumeroAleatorio()`, que utiliza o objeto `Math`.

```
function gerarNumeroAleatorio() {
    return parseInt(Math.random() * 10 + 1);
}
```

3. Interação do Usuário e Eventos

A interatividade acontece quando o usuário realiza uma ação, como clicar em um botão. No HTML, usamos o atributo `onclick` para "avisar" ao JavaScript que uma função deve ser disparada.

Capturando Valores de Input

Diferente das tags de texto (h1, p), para ler o que o usuário digitou em uma caixa de entrada, usamos a propriedade `.value`.

- **Exemplo:** `let chute = document.querySelector('input').value;`

4. Lógica de Jogo e o Tipo Booleano

O núcleo do jogo reside na comparação entre o chute do usuário e o número secreto. Essa comparação resulta em um valor **Booleano**: `true` (verdadeiro) ou `false` (falso).

Utilizamos a estrutura condicional `if/else` para guiar o fluxo do programa:

- **Se (** `if` **)** o chute for igual ao número secreto, exibimos a mensagem de vitória.
- **Caso contrário (** `else` **)**, verificamos se o número secreto é maior ou menor, fornecendo dicas ao jogador e incrementando o contador de tentativas.

5. Gerenciamento de Estado e Reinicialização

Um sistema robusto precisa ser capaz de resetar seu estado inicial. Para reiniciar o jogo, realizamos as seguintes etapas:

1. Gerar um novo número aleatório.
2. Limpar o campo de entrada.
3. Resetar o contador de tentativas para 1.
4. Exibir as mensagens iniciais.
5. Manipular atributos de botões (habilitar/desabilitar) via `document.getElementById('id').removeAttribute('disabled')`.

Glossário Técnico

Termo	Definição
Booleano	Tipo de dado que admite apenas dois valores: verdadeiro ou falso.

Case Sensitive	Característica de linguagens que diferenciam letras maiúsculas de minúsculas.
DOM	Representação estruturada da página web que permite a manipulação via script.
Parâmetro	Informação enviada para uma função para que ela possa realizar seu trabalho de forma dinâmica.
Template String	Forma de concatenar textos e variáveis usando crases (` ) e a sintaxe `${variavel}` .
Token	Palavra reservada que finaliza uma função e retorna um valor para o ponto de